A bit-identical Kokkos runtime for WAVEWATCH III: CPU results and five speed-ups

18 October 2026. Status: **WIP.** This record is for wave-modelling physicists and WW3 developers. It gives the CPU results of an optional C++/Kokkos runtime for WAVEWATCH III® (WW3), their limits, and the result of five speed-ups that do not change one output bit. One of them is a change to the Fortran of `W3SDS4` that any ST4 build can use.

Code references are `file:line`. The Fortran is pure NOAA WW3 at commit `03adda91` or the fork at commit `80644ba9`, whose `w3src4md.F90`, `w3snl1md.F90` and `w3pro3md.F90` are identical to `03adda91`'s. The C++ paths are those of the port (`kokkos/...`).

How to read the marks

Mark	Meaning
Checked (v)	Compared with a run output, a data file or the source code. The source is given.
Assumption ⚠	Not checked. A projection or an inference, not a measurement.
Position	The opinion of the author, given as a recommendation.
WIP	Work that started and is not complete.

1. Summary

- **Checked (v).** The runtime is optional. A run-time flag selects each part. With no flag, the executable runs the original Fortran.
- **Checked (v).** On the CPU, the "resident" configuration gives output files byte-identical to the Fortran of the same executable. This is true on the test cases `smoke` and `full`, at 1 to 32 threads, and on two inputs of the WW3 regtests (section 4).
- **Checked (v), single runs** ⚠**.** On one 32-thread node with a `SHRD OMPG` build, `ww3_shel` took 1.5 to 2.4 times less wall time than WW3 built from the same commit, on the `full` case (section 3). The machine was shared. No result is a median of three on an idle machine yet.
- **Checked (v).** At one thread the Kokkos code and the Fortran take the same time. The gain comes from the distribution of the work over threads.
- **Checked (v).** The gain applies to shared-memory builds only. Under `DIST MPI`, the build of GFS-Wave, the runtime keeps the Fortran path by design.
- **Checked (v).** Three of the five speed-ups are built. They keep every output bit. The DIA kernel alone is 1.93 times faster at one thread (section 6).
- **Checked (v).** A loop interchange in the Fortran `W3SDS4` keeps every output bit of WW3 (section 7). It applies to every ST4 build, `DIST MPI` included.

2. What the runtime is

WW3 integrates the action balance equation

$$\frac{\partial N}{\partial t} + \nabla_x \cdot (\dot{x}N) + \frac{\partial}{\partial k}(\dot{k}N) + \frac{\partial}{\partial \theta}(\dot{\theta}N) = \frac{S}{\sigma}, \qquad S = S_{in} + S_{nl} + S_{ds} + ...$$

with fractional steps: spatial propagation (`W3XYP3`, the third-order UQ scheme, switch `PR3 UQ`), intra-spectral propagation (`W3KTP3`), and the source terms per sea point (`W3SRCE`). The runtime keeps these steps and their arithmetic. It moves them to C++/Kokkos.

- **Checked (v).** The fork adds the switch `KOKKOS`, a branch at each call site and `bind(C)` interfaces. Each part has a flag `WW_KOKKOS_<PART>`. Unset or `0` runs the Fortran routine, which stays compiled in the same executable.
- **Checked (v).** The "resident" preset sets six parts: `SRCE` (all of `W3SRCE` for every sea point in one batched launch: `W3SIN4`, `W3SDS4`, `W3SNL1`, `W3SPR4` and the integration), `XYP3`, `KTP3`, `OUTG`, `IOPE` and `UPDV`. The spectrum `VA` stays in Kokkos memory between time steps. It goes back to the host only before a Fortran routine reads it.
- **Checked (v).** The C++ is compiled with contraction off (`-ffp-contract=off`), so no fused multiply-add changes a rounding.

3. CPU results on the `full` case

The `full` case: a 120 x 80 rectilinear grid, 9 520 sea points, `NK = 24`, `NTH = 24` (576 spectral components), 12 hours, ST4 physics with the DIA, `PR3 UQ`, switch `SHRD OMPG`. Machine: Intel Core i9-14900, 24 cores, 32 hardware threads. Compilers: gfortran and g++ 16.2.0. Kokkos 5.2.2, OpenMP backend.

Command: `bash scripts/ww3-gpu.sh --ww3-gpu full --cpu --kokkos resident`. The script builds the fork, runs it twice (all flags at 0, then the preset), then builds and runs pure NOAA WW3.

Run	Load (1 min)	NOAA WW3 (s)	Fortran, same executable (s)	Kokkos resident (s)	Ratio to NOAA WW3
17 Oct, fork `769c44f0`	35.9 to 46.5	43.09	41.92	17.98	2.40
18 Oct, morning	not recorded	24.81	23.38	15.51	1.60
18 Oct, quietest	3.5 / 11.8	not run	26.31	14.36	1.83 to the same executable
18 Oct, final code	not recorded	32.06	30.86	18.36	1.75

- **Checked (v).** In every run, the Kokkos output is byte-identical (`cmp`) to the Fortran of the same executable. Against NOAA WW3, `out_grd.ww3` and `mod_def.ww3` are byte-identical. `ww3.field.nc` differs only in the global attribute `WAVEWATCH_III_switches`, which ends with `KOKKOS` in the fork. `nccmp-tol` finds `max|d| = 0` on `hs`.
- **Assumption ⚠.** The same NOAA build took 24.8 s in one run and 43.1 s in another. The load of other jobs on the machine explains this spread. A number for a table needs the median of three runs on an idle machine.

Thread scaling (18 October, one run each, shared machine ⚠)

Threads	Kokkos resident (s)	Fortran, same executable (s)	Fortran / Kokkos
1	174.78	174.27	1.00
8	47.35	56.13	1.19
16	29.21	36.46	1.25
32	14.36	26.31	1.83

Checked (v). The `out_grd.ww3` of every row is identical to the Fortran's. The bits do not depend on the number of threads.

4. Bit parity: what is shown and what is not

- **Checked (v).** Two inputs of the WW3 regtests pass with the resident preset: `ww3_tp2.15` (`test.comp` finds 0 of 23 files different, `nccmp -tol` PASS) and `ww3_ts2 ST4_PR3_UQ` (0 of 22). Both runs used the source-term switches of the test build (`BT1 DB1 LN1 MLIM`), because no `ST4 NL1` switch file of the regtests has this set.
- **Checked (v).** The DIA (`W3SNL1`) alone passes the `NL1` entries of the regtest matrix.
- **Not shown.** Parity with the Intel compiler; parity on a GPU (judged by tolerance); `ww3_multi` with the resident spectrum; restart reproduction.

5. Where the CPU time goes

The quietest run of section 3, 32 threads, from the runtime's timers and gdb stack samples. The split of the `W3SRCE` kernel into terms comes from sample shares, so those rows are derived ⚠.

Rank	Part	s	Share of wall
1	`W3SNL1` (DIA)	6.37	44.4 %
2	`W3SDS4`	3.10	21.6 %
3	`W3SIN4`	1.41	9.8 %
4	`W3SRCE` integration	1.12	7.8 %
5	`W3XYP3`	1.09	7.6 %
6	`W3KTP3`	0.39	2.7 %
	Fortran outside Kokkos, I/O, set-up	0.22	1.5 %
	VA copies	0.002	0.0 %

Why the Fortran is slower at 32 threads.

- **Checked (v).** In NOAA WW3 with `SHRD OMPG`, the loop over the spectral components for spatial propagation is serial (`w3wavemd.F90:1944`, `DO ISPEC=1, NSPEC`). `W3GATH` and `W3SCAT` have no OpenMP directive (`w3wavemd.F90:3024-3644`). Each call of `W3XYP3` opens its own parallel regions over the sea points (`w3pro3md.F90:978`, `:1077`, `:1106`, `:1126`).
- **Checked (v).** The Kokkos `W3XYP3` propagates all components in one launch, in parallel over the components.
- **Assumption ⚠.** At one thread, the Kokkos parts other than `W3SRCE` take 18.3 s. If the Fortran spends the same time there and its `W3SRCE` loop scales like the Kokkos one, the rest of the Fortran takes about 14 s at 32 threads: a speed-up of about 1.3 from 1 to 32 threads. A timing per region of the Fortran at 32 threads can confirm or reject this.
- **Position.** If this is correct, a parallel loop over the components in the Fortran `SHRD` path gives part of the gain to OpenMP builds of WW3 without Kokkos. Each component is independent, so the bits do not change.

6. Five speed-ups that keep every bit

Each item must make the resident mode faster, keep every output bit, and give WW3 a part that it can use alone. Three items are built. Two items needed no change.

Item	Target	Status	Bits	Result
6.1 Thread placement	all launches	no change	Checked (v)	not measured ⚠
6.2 `-fno-math-errno`	source terms	built	Checked (v)	not separated ⚠
6.3 W3XYP3 load balance	rank 5	no change	Checked (v)	already in place
6.4 W3SDS4 saturation integral	rank 2	built	Checked (v)	loop 1.7 to 1.9 times faster
6.5 DIA halo layout	rank 1	built	Checked (v)	kernel 1.93 times faster, 1 thread

Checked (v), for all items. With the final code, `full` and `smoke` give model files byte-identical to the Fortran of the same executable, and `nccmp-tol` passes against NOAA WW3 with `max|d| = 0`. About 40 runs of `full` during the work, at 1 and 32 threads, all gave an `out_grd.ww3` identical to the Fortran's.

6.1 Thread placement

- **Checked (v).** The run script passes `OMP_PROC_BIND` and `OMP_PLACES` from the environment to both runs, so no code change is necessary.
- **Checked (v).** The output is byte-identical at 1, 8, 16 and 32 threads. Placement changes which thread runs a point, not the arithmetic.
- **WIP.** Compare `OMP_PLACES=cores OMP_PROC_BIND=spread` with the default on an idle machine. On a shared machine, the measurement cannot rank them.

6.2 `-fno-math-errno`

- **Change.** The C++ library uses gfortran's default. Before, `g++` added a branch to glibc's `sqrtf` for a negative argument, only to set `errno`.
- **Bits. Checked (v).** IEEE 754 square root is correctly rounded, so `sqrtss` and glibc's `sqrtf` give the same value. All runs stayed byte-identical.
- **Speed. Assumption ⚠.** Not separated from the other changes.

6.3 W3XYP3 load balance

- **Checked (v).** The list of components is in ascending `ISPEC`. So the lowest frequencies, which need the most CFL (Courant-Friedrichs-Lewy) sub-steps, start first. This is already the best order for a dynamic schedule.
- **Assumption ⚠.** The Kokkos chunk for 576 items on 32 threads is one item. This was read from the Kokkos rule, not printed. No bit-neutral change was left.

6.4 The saturation integral of W3SDS4

The original statement (`w3src4md.F90:2281-2285`) sums 23 products for each direction: `BTH(IS) = DOT_PRODUCT(SATWEIGHTS(:,ITH), A(IS0+SATINDICES(:,ITH))) * FACSAT`. One sum is a chain of 23 dependent additions.

- **Change.** The loop over the weight index J is outside and the loop over ITH is inside. The C++ reads transposed copies of SATWEIGHTS and SATINDICES, with the direction index fastest, so the inner loop reads contiguous memory.
- **Bits. Checked (v).** Each BTH(IS) still starts from 0, adds its 23 products in the order J = 1, ..., 23, and is then multiplied by FACSAT.
- **Speed. Checked (v).** In an isolated test of this loop (best of 7 x 20 000 calls, NK = 24, NTH = 24), one sea point took 5.5 to 5.7 µs with the original order, 5.1 µs with the interchange on the original tables, and 3.0 to 3.3 µs with the transposed tables. The transposed tables give most of the gain.
- **Excluded.** 12 of the 23 weights are zero for SSDSDTH = 80. To skip them changes the result when one A is infinite or NaN (0 x Inf is NaN), so this is not a bit-neutral change.

6.5 The DIA with a direction halo

Sections 3 and 4 of W3SNL1 read the spectrum at the quadruplet bins through the index tables IP, IM and IC (w3snl1md.F90:378-440). With the bins vectorised, each read is a gather.

- **Checked (v).** Each table entry is IT(ITH) + (IF(IFR)-1)*NTH: a direction shift with a wrap, and a map of the frequency row (w3snl1md.F90:667-727).
- **Change.** The kernel copies the arrays into rows with a halo of the wrapped neighbour directions on both sides. A read is then the same row and a fixed shift for all directions of a frequency, so the loads are contiguous. The set-up compares every table entry with the row and the shift. If one entry is different, the kernel keeps the gathers.
- **Bits. Checked (v).** The values read and the arithmetic are those of the gather form. Only the location of the copies changes.
- **Speed. Checked (v).** The DIA kernel alone (1 000 sea points, NK = 25, NTH = 24, median of 7 alternated runs, machine load 8.4 to 8.9):

Threads	Before (ms per call)	After (ms per call)	Ratio
1	12.31	6.39	1.93
32	2.50	2.11	1.19

- **Assumption ⚠.** Inside the model, with the halo switched on and off in one executable, the W3SRCE kernel took 7 % to 15 % less time with the halo (32 threads, load 31 to 57). The whole ww3_shel medians, before and after all changes, were 17.67 s and 17.09 s (load 8 to 38). That difference is inside the spread of the runs, so the end-to-end gain is not measured yet.

7. The Fortran change for WW3: W3SDS4

Checked (v). This change was built and run, not only proposed. It replaces lines 2281-2285 of w3src4md.F90 and adds one integer declaration (INTEGER :: JSAT):

```
! partial integration: JSAT outer, ITH inner. Each BTH(IS) adds its
! products in the order of DOT_PRODUCT, from 0, then times FACSAT.
DO ITH=1,NTH
  BTH(IS0+ITH)=0.
END DO
DO JSAT=1,SIZE(SATWEIGHTS,1)
  DO ITH=1,NTH
    BTH(IS0+ITH)=BTH(IS0+ITH)                                    &
         +SATWEIGHTS(JSAT,ITH)*A(IS0+SATINDICES(JSAT,ITH))
  END DO
END DO
DO ITH=1,NTH
```

```
      BTH(IS0+ITH)=BTH(IS0+ITH)*FACSAT
    END DO
```

- **Static check. Checked (v).** In gfortran's optimised intermediate code at `-O3` with the build's flags (`-fdump-tree-optimized`), the original sets `val = 0.0`, adds each `weight*A` in index order, and returns `facsat*val`. The new loop sets `BTH = 0.0`, adds the same products in the same order, and multiplies by `facsat`. IEEE 754 addition and multiplication are commutative, so each result has the same bits.
- **Run check. Checked (v).** The original and the changed WW3 were built from commit `03adda91` with the same switch and flags. On `full`, at 32 threads and twice at one thread, the changed build wrote an `out_grd.ww3` byte-identical to the original's.
- **Control. Checked (v).** A build with `SRHS = SRHS * 1.0001` in `W3SDS4` changes `out_grd.ww3` (first difference at byte 267 482). So the comparison detects a change in `W3SDS4`. The control line must come before the whitecap-output test of `W3SDS4`: the usual path returns there, and a line before the last `RETURN` of the routine does not run.
- **Speed. Assumption ⚠.** The CPU time at one thread was 296.4 s and 280.8 s for the original and 295.8 s and 196.2 s for the changed build. The machine load explains more of this spread than the change does. In C++, most of the gain needed the transposed tables (section 6.4).
- **Position.** Offer the interchange together with transposed copies of `SATWEIGHTS` and `SATINDICES`, built once in `INSIN4`. Measure it in an MPI build of GFS-Wave's grid before a speed is claimed.

8. Limits

- **Checked (v).** GFS-Wave uses `DIST MPI`. Under MPI the resident parts `XYP3`, `OUTG`, `IOPE` and `UPDV` refuse to run by design, and the runtime is single node. Do not read the results of section 3 as a speed-up of GFS-Wave.
- **Checked (v).** The Kokkos `W3SRCE` translates one source-term set (`ST4 NL1` with `BT1 DB1 LN1 MLIM`). Other sets run the Fortran.
- **Assumption ⚠.** Bit parity is shown for gfortran and g++ on x86-64 only.
- **WIP.** On a GPU, the device must give glibc's bits for `expf`, `logf` and `powf`. Device copies of glibc's functions exist; no device run has compared them yet.

9. Before a speed is final

1. On an idle machine, run `bash scripts/ww3-gpu.sh --ww3-gpu full --cpu --kokkos resident --repeat 3`, before and after the changes of section 6. Record the medians.
2. Time the Fortran per region at 32 threads, to confirm or reject the assumption of section 5.
3. Compare the thread placements of section 6.1 on the idle machine.
4. Time the Fortran change of section 7, with and without transposed tables, on an idle machine and in an MPI build.

10. How to repeat

What	Command
Parity and wall time, CPU	`bash scripts/ww3-gpu.sh --ww3-gpu full --cpu --kokkos resident`
The same on the point case	`bash scripts/ww3-gpu.sh --ww3-gpu smoke --cpu --kokkos resident`
Per-part timers	add `WW_KOKKOS_TIMERS=1` to the Kokkos run, or `--timers`

What	Command
DIA kernel alone	`OMP_PROC_BIND=false OMP_NUM_THREADS=<t> kokkos/build/openmp-release/tests/ww_bench_snl1`
Tests of the port’s tools	`just kokkos-test serial-debug`, `just kokkos-test openmp-release`